# Установка и настройка Neovim с нуля (Windows 11)

Пошаговая инструкция, как повторить эту конфигурацию на другом компьютере · октябрь 2026

## Что получится

- Neovim 0.12 с менеджером плагинов lazy.nvim.
- Подсветка кода (Treesitter), автодополнение (blink.cmp), поиск файлов и текста (Telescope), файловое дерево (nvim-tree), тема tokyonight, значки (Nerd Font).
- Подсказки, ошибки и переход к определению для Python (Pyright), Rust (rust-analyzer) и PowerShell (PowerShell Editor Services).
- Локальный ИИ-помощник: Ollama, модель qwen2.5-coder:7b и плагин codecompanion.nvim.

Перед началом: нужен интернет. Для локальной модели желательна видеокарта с 8 ГБ памяти, без неё модель работает на процессоре и заметно медленнее. Все команды выполняются в PowerShell. После установки программ закрой PowerShell и открой заново, чтобы обновились пути.

## Шаг 1. Установить программы

```
winget install Neovim.Neovim
winget install Git.Git
winget install Python.Python.3.12
winget install OpenJS.NodeJS
winget install BurntSushi.ripgrep.MSVC
winget install tree-sitter.tree-sitter-cli
winget install DEVCOM.JetBrainsMonoNerdFont
winget install Rustlang.Rustup
winget install Ollama.Ollama
```

Заметки:

- Если winget не находит пакет, найди его командой `winget search имя`: идентификаторы иногда меняются.
- Node.js нужен для Pyright, Python нужен для разбора Python-кода, Rust ставится через rustup.

- Для сборки парсеров Treesitter нужен компилятор C. На исходном компьютере стоит Visual Studio. Если `:TSInstall` пожалуется на отсутствие компилятора, поставь Build Tools для Visual Studio с компонентом «Разработка классических приложений на C++» или другой компилятор (например, zig).
- Буфер обмена Neovim обычно работает через win32yank, который идёт в комплекте. Проверка: `:checkhealth vim.provider`.

## Шаг 2. Выбрать шрифт в Windows Terminal

1. Нажми `Ctrl+,` (откроются настройки).
2. Слева в разделе «Профили» выбери «По умолчанию», затем вкладку «Оформление».
3. В поле «Шрифт» выбери JetBrainsMono Nerd Font и нажми «Сохранить».

Если шрифта нет в списке, закрой Windows Terminal полностью и открой заново. Без этого шага вместо значков будут ромбики.

## Шаг 3. Перенести конфиг

Конфиг лежит в `$env:LOCALAPPDATA/nvim`. Если эта папка уже есть, переименуй её: `Rename-Item $env:LOCALAPPDATA/nvim nvim.old`.

**Вариант А: из GitHub.**

```
git clone АДРЕС_РЕПОЗИТОРИЯ $env:LOCALAPPDATA/nvim
```

**Вариант Б: без GitHub.** Скопируй папку `nvim` со старого компьютера (флешка, архив) в `$env:LOCALAPPDATA`. Проверка: `Get-ChildItem $env:LOCALAPPDATA/nvim` должна показать `init.lua`, `lua` и `lazy-lock.json`.

**Вариант В: вручную.** Создай файлы по приложению в конце этой инструкции.

Файл `lazy-lock.json` закрепляет точные версии плагинов, поэтому на новом компьютере всё ставится ровно таким, как работает сейчас.

## Шаг 4. Первый запуск

```
nvim
```

lazy.nvim скачает сам себя и все плагины. Появится окно со списком, подожди, пока загрузка закончится, и закрой окно клавишей `q`. Плагин blink.cmp при первом запуске скачивает готовый бинарный файл (нужен интернет). Если

появились красные ошибки, закрой Neovim и запусти заново.

## Шаг 5. Парсеры подсветки

В Neovim выполни:

```
:TSInstall lua python rust
```

Дай каждому языку 20-40 секунд. Проверка: `:checkhealth nvim-treesitter`. В списке Installed languages должны быть `lua`, `python` и `rust`.

## Шаг 6. Языковые серверы

**Python (Pyright):**

```
npm install -g pyright
pyright --version
```

**Rust (rust-analyzer):**

```
rustup component add rust-analyzer
rust-analyzer --version
```

**PowerShell (PowerShell Editor Services).** Скачать и распаковать:

```
$dir = Join-Path $env:LOCALAPPDATA 'nvim-data/pses'
New-Item -ItemType Directory -Force $dir | Out-Null
Invoke-WebRequest
'https://github.com/PowerShell/PowerShellEditorServices/releases/latest/downl
oad/PowerShellEditorServices.zip' -OutFile (Join-Path $dir 'pses.zip')
Expand-Archive (Join-Path $dir 'pses.zip') -DestinationPath $dir -Force
Test-Path (Join-Path $dir 'PowerShellEditorServices/Start-
EditorServices.ps1')
```

Последняя команда должна вывести `True`.

Важно: в `lsp.lua` сервер запускается через `powershell` (встроенный Windows PowerShell 5.1), потому что PowerShell 7 из Microsoft Store Neovim запустить не может. Если PowerShell 7 установлен обычным установщиком, можно заменить `powershell` на `pwsh`.

## Шаг 7. Локальная модель

Ollama после установки запускается сама (значок ламы в трее). Затем:

```
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder:7b 'Привет'
ollama ps
```

Модель занимает около 4,7 ГБ. В колонке PROCESSOR команды `ollama ps` должно быть `100% GPU`. Плагин настроен на контекст 16384 токена, это около 5,5 ГБ видеопамяти. Если видеопамяти меньше 8 ГБ, уменьши `num_ctx` в `lazy.lua` до 8192.

## Шаг 8. Punto Switcher (если он установлен)

Punto Switcher переключает раскладку на лету и ломает ввод команд. Правый клик по значку в трее, затем Настройки, раздел «Программы-исключения», добавить `WindowsTerminal.exe`.

## Шаг 9. Проверка

- `:checkhealth` не показывает красных ошибок, а `:checkhealth vim.provider` показывает Python, Node, Perl и Ruby как Disabled.
- Файл `test.py` с одной строкой `print(abc)`: справа появляется ошибка `abc is not defined`.
- Работают `Ctrl+p` (поиск файла), `Ctrl+f` (поиск текста), `Ctrl+n` (дерево).
- Набор `pri` в Python-файле открывает меню автодополнения.
- Пробел a открывает чат, затем `i`, вопрос и `Ctrl+s`: приходит ответ модели.

## Приложение. Файлы конфигурации

Папка: `$env:LOCALAPPDATA/nvim`.

**init.lua**

```
vim.g.loaded_perl_provider = 0
vim.g.loaded_ruby_provider = 0
vim.g.loaded_node_provider = 0
vim.g.loaded_python3_provider = 0

require('config.options')
require('config.lazy')
require('config.lsp')
```

**lua/config/options.lua.** В исходном файле уже были базовые настройки редактора (при ручной установке можно начать со строки `vim.opt.number = true`). В конец добавлено:

```
vim.api.nvim_create_autocmd('FileType', {
  pattern = { 'python', 'lua', 'rust' },
  callback = function() pcall(vim.treesitter.start) end,
})

vim.keymap.set('i', 'jk', '<Esc>')

vim.g.mapleader = ' '
vim.keymap.set({ 'n', 'v' }, '<leader>a', '<cmd>CodeCompanionChat
Toggle<cr>')
vim.keymap.set({ 'n', 'v' }, '<leader>i', ':CodeCompanion ')
vim.keymap.set('v', '<leader>c', '<cmd>CodeCompanionChat Add<cr>')
```

### lua/config/lazy.lua

```
local lazypath = vim.fn.stdpath('data') .. '/lazy/lazy.nvim'
if not (vim.uv or vim.loop).fs_stat(lazypath) then
  vim.fn.system({ 'git', 'clone', '--filter=blob:none', '--branch=stable',
    'https://github.com/folke/lazy.nvim.git', lazypath })
end
vim.opt.rtp:prepend(lazypath)

require('lazy').setup({
  spec = {
    { 'nvim-treesitter/nvim-treesitter', build = ':TSUpdate' },
    { 'saghen/blink.cmp', version = '1.*', opts = {} },
    {
      'nvim-telescope/telescope.nvim',
      dependencies = { 'nvim-lua/plenary.nvim' },
      keys = {
        { '<C-p>', '<cmd>Telescope find_files<cr>' },
        { '<C-f>', '<cmd>Telescope live_grep<cr>' },
      },
    },
    {
      'folke/tokyonight.nvim',
      lazy = false,
      priority = 1000,
      config = function() vim.cmd.colorscheme('tokyonight') end,
    },
    {
      'nvim-tree/nvim-tree.lua',
```

```
      dependencies = { 'nvim-tree/nvim-web-devicons' },
      keys = { { '<C-n>', '<cmd>NvimTreeToggle<cr>' } },
      opts = {},
    },
    {
      'olimorris/codecompanion.nvim',
      dependencies = {
        'nvim-lua/plenary.nvim',
        'nvim-treesitter/nvim-treesitter',
      },
      cmd = { 'CodeCompanion', 'CodeCompanionChat', 'CodeCompanionActions' },
      opts = {
        interactions = {
          chat = { adapter = { name = 'ollama', model = 'qwen2.5-coder:7b' }
},
          inline = { adapter = { name = 'ollama', model = 'qwen2.5-coder:7b'
} },
        },
        adapters = {
          http = {
            ollama = function()
              return require('codecompanion.adapters').extend('ollama', {
                schema = {
                  num_ctx = { default = 16384 },
                  keep_alive = { default = '30m' },
                },
              })
            end,
          },
        },
      },
    },
  },
  rocks = { enabled = false },
})
```

## lua/config/lsp.lua

```
vim.lsp.config('pyright', {
  cmd = { 'pyright-langserver', '--stdio' },
  filetypes = { 'python' },
  root_markers = { 'pyrightconfig.json', 'pyproject.toml', 'setup.py',
'requirements.txt', '.git' },
})
vim.lsp.enable('pyright')
```

```
vim.diagnostic.config({ virtual_text = true })

vim.api.nvim_create_autocmd('LspAttach', {
  callback = function(args)
    vim.keymap.set('n', 'gd', vim.lsp.buf.definition, { buffer = args.buf })
  end,
})

local pses = vim.fs.normalize(vim.fn.stdpath('data') .. '/pses')
local cache = vim.fs.normalize(vim.fn.stdpath('cache'))
local pses_cmd = string.format(
  [[& '%s/PowerShellEditorServices/Start-EditorServices.ps1' -
BundledModulesPath '%s' -LogPath '%s/powershell_es.log' -SessionDetailsPath
'%s/powershell_es.session.json' -FeatureFlags @() -AdditionalModules @() -
HostName nvim -HostProfileId 0 -HostVersion 1.0.0 -Stdio -LogLevel Normal]],
  pses, pses, cache, cache
)

vim.lsp.config('powershell_es', {
  cmd = { 'powershell', '-NoLogo', '-ExecutionPolicy', 'Bypass', '-
NoProfile', '-Command', pses_cmd },
  filetypes = { 'ps1' },
  root_markers = { 'PSScriptAnalyzerSettings.psd1', '.git' },
})
vim.lsp.enable('powershell_es')

vim.lsp.config('rust_analyzer', {
  cmd = { 'rust-analyzer' },
  filetypes = { 'rust' },
  root_markers = { 'Cargo.toml', 'rust-project.json', '.git' },
})
vim.lsp.enable('rust_analyzer')
```

## Резервная копия конфига

Сохранить версию (уже сделано один раз):

```
cd $env:LOCALAPPDATA/nvim
git add .
git commit -m 'Описание изменений'
```

Если Git не знает, кто ты: `git config --global user.name 'Имя'` и `git config --global user.email 'почта'`.

Выложить на GitHub (по желанию): создай на github.com пустой закрытый репозиторий, затем выполни

```
git remote add origin АДРЕС_РЕПОЗИТОРИЯ
git push -u origin master
```

В копию попадают только твои файлы и `lazy-lock.json`. Сами плагины в неё не входят: lazy скачает их при первом запуске. Не клади в репозиторий пароли и ключи доступа (сейчас их в конфиге нет).